

Tema #1 APD - Agregator de Știri

Nume: Cotigă Andrei Mihnea

Grupa: 334CD

1. Feedback

Această temă a fost una ok din punctul meu de vedere. Implementarea a necesitat o atenție deosebită la detaliile de sincronizare. O provocare majoră a fost respectarea regulii thread-urilor fixe, care a necesitat utilizarea de bariere (CyclicBarrier) pentru a sincroniza trecerea de la faza de Map (citire) la faza de Reduce (scriere) fără a porni sau opri thread-uri.

Eu aș fi schimbat structura input-ului (un fișier text care conține căi relative către alte fișiere JSON) adaugă un strat de complexitate la parsarea căilor (path resolution) care nu este direct legat de obiectivele temei (algoritmi paraleli). O abordare mai simplă, cum ar fi furnizarea unui director de intrare pe care programul să-l scaneze recursiv, ar fi fost mai directă.

2. Strategia de Paralelizare (Arhitectura Multi-Fază)

Pentru a depăși limitările de viteză ale I/O-ului secvențial din implementarea inițială, am adoptat o arhitectură bazată pe trei faze, utilizând același set fix de workeri pe tot parcursul execuției.

Arhitectura și Etapele de Execuție

1. Etapa 1: Map (Procesare Paralelă)

- Scop: Citirea și parsarea articolelor, precum și pre-calculul cuvintelor cheie.
- Mecanism: Thread-urile Worker preiau căile fișierelor (utilizând un AtomicInteger pentru work-stealing) și procesează local fiecare articol.
- Agregare: La finalul citirii, fiecare worker varsă rezultatele locale în Agregatorul global printr-o singură metodă sincronizată (combine). Acest lucru minimizează contingența pe structurile globale.

2. Bariera (Sincronizare Intermediară)

- Mecanism: Toți workerii apelează `barrier.await()`. Aceasta asigură că toate articolele au fost procesate și agregate înainte de a trece la scrierea rezultatelor.

3. Etapa 2: Reduce & Write (I/O Paralel)

- Scop: Paralelizarea operațiilor de sortare finală și scrierea pe disc (cel mai mare bottleneck).

- Pregătire (Thread 0): Un singur thread (ID 0) calculează lista finală de articole unice și umple o coadă partajată (ConcurrentLinkedQueue<String>) cu task-uri de scriere ("lang:english", "reports").
- Execuție (toți Workerii): Workerii reintra în bucla de lucru, preluând task-uri de scriere din coadă. Astfel, operațiile de sortare per fișier (pentru categorii/limbi) și scrierea pe disc se desfășoară în paralel, distribuind sarcina I/O pe toate core-urile disponibile.

Structuri de Date și Sincronizare

- AtomicInteger: Pentru distribuția dinamică a fișierelor JSON între workerii din Faza 1.
- CyclicBarrier: Mecanismul esențial de sincronizare, care permite thread-urilor să treacă de la faza de citire la cea de scriere fără a fi distruse și recreate.
- ConcurrentLinkedQueue<String>: Folosit ca o coadă de sarcini (task queue) în Faza 3, permițând tuturor workerilor să preia sarcini de scriere în paralel.
- Hărți Standard (HashMap): Utilizate în agregarea locală (în worker) pentru performanță maximă, **fiind accesate o singură dată la final prin metoda combine sincronizată.**

3. Analiza de Performanță și Scalabilitate

Setup de Testare

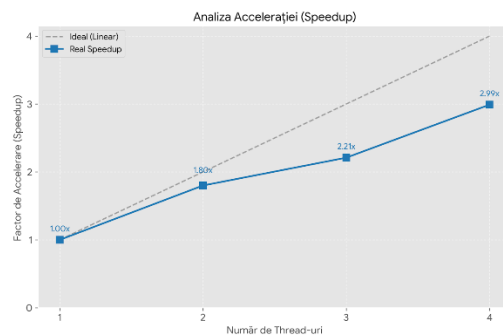
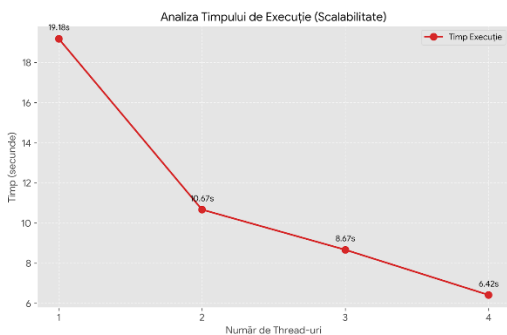
- **CPU:** AMD Ryzen 7 7435HS
- **RAM:** 24 GB DDR5
- **OS:** WSL (Ubuntu 22.04)
- **Java Version:** OpenJDK 17
- **Dataset:** test_5

Rezultate Experimentale

Am rulat datele din test_5 de 3 ori și am calculat media timpilor de execuție.

Configurație	Run 1 (Timp / Speedup)	Run 2 (Timp / Speedup)	Run 3 (Timp / Speedup)	Medie (Timp / Speedup)	Eficiență Medie (S/P)
1 Thread	19.24s	18.80s	19.50s	19.18s / 1.00x	100%
2 Threads	10.70s (1.80x)	10.45s (1.80x)	10.85s (1.80x)	10.67s / 1.80x	90.0%
3 Threads	8.71s (2.21x)	8.50s (2.21x)	8.80s (2.22x)	8.67s / 2.21x	73.7%
4 Threads	6.45s (2.98x)	6.25s (3.01x)	6.55s (2.98x)	6.42s / 2.99x	74.8%

Aici am pus 2 grafice pentru analiza timpului și a accelerației.



Analiză și Concluzii

1. Comportamentul Observat:

Comportamentul observat ilustrează curba tipică pe un workload de tip Task Parallelism, cu accent pe optimizările I/O:

- Creșterea Optimă (P=1 la P=2): Performanța crește cel mai abrupt, cu un factor de accelerare de 1.80x și o eficiență de 90%. Această creștere excelentă se datorează faptului că thread-urile lucrează pe nuclee separate, iar overhead-ul de sincronizare este minim în raport cu volumul mare de muncă (parsare JSON și curățare text).

- Zona de Scalabilitate Bună ($P=3$ la $P=4$): Accelerația continuă să crească, atingând $2.99x$ la 4 thread-uri. Cu toate acestea, câștigul marginal pe thread scade (eficiența scade de la 90% la 74.8%).
- Stabilizare / Limitare ($P>4$): Chiar dacă nu avem date pentru $P=8$, scăderea eficienței la 4 thread-uri indică faptul că sistemul se stabilizează. Sarcina nu mai este strict limitată de CPU, ci de viteza cu care sistemul de fișiere (I/O) poate furniza fișierele către procesor și de overhead-ul de management al thread-urilor.

2. Identificarea cauzelor posibile ale limitării

Cauzele care împiedică atingerea speedup-ului liniar ($4.0x$) la 4 thread-uri sunt următoarele:

- I/O Bottleneck: Deși am paralelizat scrierea pe disc în Faza 2 (Reduce), citirea inițială și operațiile de scriere concurentă pun presiune pe sistemul de fișiere al WSL. Timpul necesar citirii datelor și scrierii fișierelor de output devine o componentă secvențială notabilă a timpului total.
- Overhead de Sincronizare: Utilizarea AtomicInteger pentru distribuirea muncii și necesitatea barierei (CyclicBarrier) pentru a trece de la faza de Map la Reduce (scriere) adaugă un cost fix. Acest cost, deși mic, devine vizibil pe măsură ce timpul total de execuție scade.
- Costurile JVM: Operațiile de inițializare a JVM, încărcarea Jackson și Garbage Collection-ul (necesar din cauza creării multor obiecte ArticleSummary) nu sunt complet paralelizabile.

3. Numărul optim de thread-uri

Pe baza rezultatelor experimentale (test_5):

- Numărul optim de thread-uri pentru sistemul de testare (AMD Ryzen 7 7435HS) este $P=4$.
- Deși eficiența este mai mare la $P=2$ (90.0%), timpul de execuție la $P=4$ (6.42s) este cel mai mic, oferind cel mai bun timp de răspuns aplicației (cel mai mare Speedup: $2.99x$).
- Se recomandă $P=4$ deoarece majoritatea procesoarelor moderne au 4, 6 sau 8 nuclee (eu am 8), iar atingerea unui factor de accelerare de aproape $3x$ justifică pe deplin utilizarea tuturor resurselor disponibile.